

BLUETOOTH CONTROLLED SERVO GRIPPER ARM

Minor Project report submitted in partial fulfilment of the requirements for
the award of the Degree of

Bachelor of Technology
in
***Electronics & Communication Engineering/Robotics and Artificial
Intelligence (2nd Year)***

BY

HARSH RAJ ***2410090005***

NISHANT SINGH ***2410090019***

SHISHIR GUPTA ***2410090012***

RUDRA SINGH ***2410090009***

Under the guidance of:
Mr./Ms./Dr. Faculty Name
Assistant Professor/Associate Professor



Department of Electrical and Electronics Engineering
School of Engineering, IILM University, Greater Noida
Gautam Buddha Nagar, Uttar Pradesh
APRIL 2026 (2025-26)

DECLARATION

I/We hereby declare that this minor project submission is my/our own work and that, to the best of my/our knowledge and belief, it contains no material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning.

Signature:

HARSH RAJ

(241009005)

Signature:

SHISHIR GUPTA

(2410090012)

Signature:

NISHANT SINGH

(2410090019)

Signature:

RUDRA SINGH

2410090009

This is to certify that the minor project report entitled "**Bluetooth Controlled Servo Gripper Arm**" being submitted by:

RUDRA SINGH ***2410090009***

The result embodied in this minor project report has not been submitted to any other University or Institute for the award of any other Degree or Diploma.

Mr./Ms./Dr. XYZ
Project Guide
Assistant/Associate
Professor

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the project undertaken during B.Tech. second year. We owe special debt of gratitude to Dr. CHANDAN CHOUBEY (project guide), Associate Professor, Department of Electrical and Electronics Engineering, School of Engineering, IILM University, Greater Noida for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us.

We also take the opportunity to acknowledge the contribution of Dr. Vanya Arun, Associate Professor, and Head of the Department of Electrical and Electronics Engineering, School of Engineering, IILM University, Greater Noida for her full support and assistance during the development of the project.

We take this opportunity to convey our heartfelt gratitude to our project coordinator, Mr. Awadhesh Kumar Maurya, Assistant Professor, Department of Electrical and Electronics Engineering, School of Engineering, IILM University, Greater Noida for their exceptional guidance, encouragement, and whole-hearted cooperation. Their technical inputs and timely suggestions played a crucial role in bringing this project to fruition.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members (Names of all the faculty members) of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

HARSH RAJ ***2410090005***

NISHANT SINGH ***2410090019***

SHISHIR GUPTA ***2410090012***

RUDRA SINGH ***2410090009***

<i>Title</i>	<i>Page No.</i>
	vi-vii
LIST OF FIGURES	viii
ABSTRACT	1–5
Chapter 1: Introduction	1
1.1: PROBLEM STATEMENT	2
1.2: LITERATURE SURVEY	3
1.3: SYSTEM OVERVIEW	4
1.4: COMPONENTS USED	4
1.4.1: Arduino UNO	4
1.4.2: PCA9685 Servo Driver	4
1.4.3: Servo Motors	5
1.4.4: HC-05 Bluetooth Module	5
1.4.5: Buck Converter	5
1.4.6: Gripper Mechanism	6–9
Chapter 2: CIRCUIT DESIGN AND IMPLEMENTATION	6
2.1: CIRCUIT DIAGRAM DESCRIPTION	7
2.2: ARDUINO CODE	9
2.3: WORKING PRINCIPLE	10
CHAPTER 3: ADVANTAGES AND DISADVANTAGES	11
CHAPTER 4: APPLICATIONS	12
CHAPTER 5: SOFTWARE USED	13
CONCLUSION AND FUTURE SCOPE OF WORK	14
REFERENCES	

LIST OF FIGURES

Fig. No.	Title	Page
Fig. 1.1	Arduino UNO Microcontroller Board	4
Fig. 1.2	PCA9685 16-Channel Servo Driver Module	4
Fig. 1.3	MG996R Metal-Gear Servo Motor	4
Fig. 1.4	HC-05 Bluetooth Module	5
Fig. 1.5	LM2596 Buck Converter Module	5
Fig. 1.6	Two-Finger Parallel Gripper Mechanism	5
Fig. 2.1	Complete Circuit Schematic	6
Fig. 2.2	Arduino UNO – PCA9685 I2C Connection	6
Fig. 2.3	HC-05 Bluetooth Module Wiring	7
Fig. 2.4	Buck Converter Power Supply Circuit	7
Fig. 2.5	Arduino IDE – Servo Control Sketch	8
Fig. 2.6	Assembled Gripper Arm Prototype	9

COMPONENT FIGURES

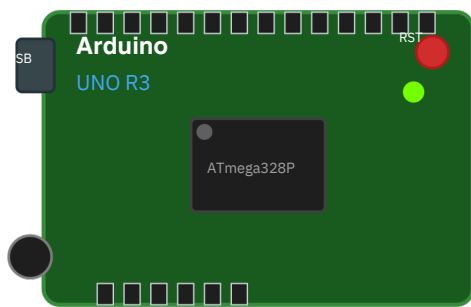


Fig. 1.1 – Arduino UNO R3 (ATmega328P)

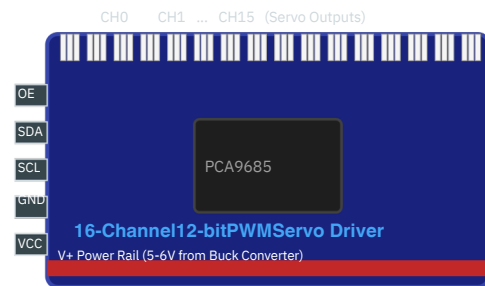


Fig. 1.2 – PCA9685 16-Ch PWM Servo Driver

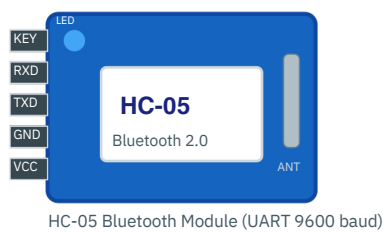


Fig. 1.4 – HC-05 Bluetooth Module

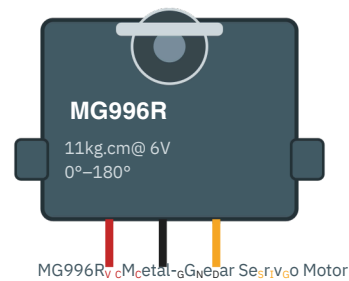


Fig. 1.3 – MG996R Metal-Gear Servo (x6)

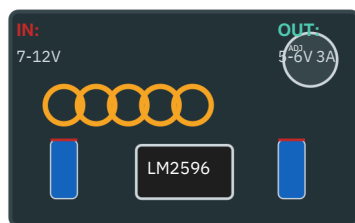


Fig. 1.5 – LM2596 DC-DC Buck Converter

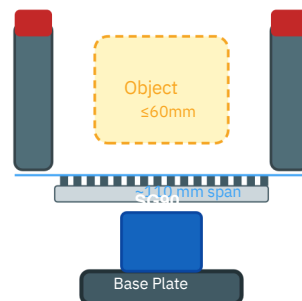


Fig. 1.6 – Two-Finger Parallel Gripper (SG90 driven)

COMPLETE CIRCUIT SCHEMATIC

The schematic below shows all subsystems and their interconnections: battery and buck converter power supply, Arduino UNO microcontroller, PCA9685 PWM servo driver (I2C bus), HC-05 Bluetooth UART interface with voltage divider protection, and seven servo channels (MG996R + SG90 gripper). All grounds are connected to a common GND bus.

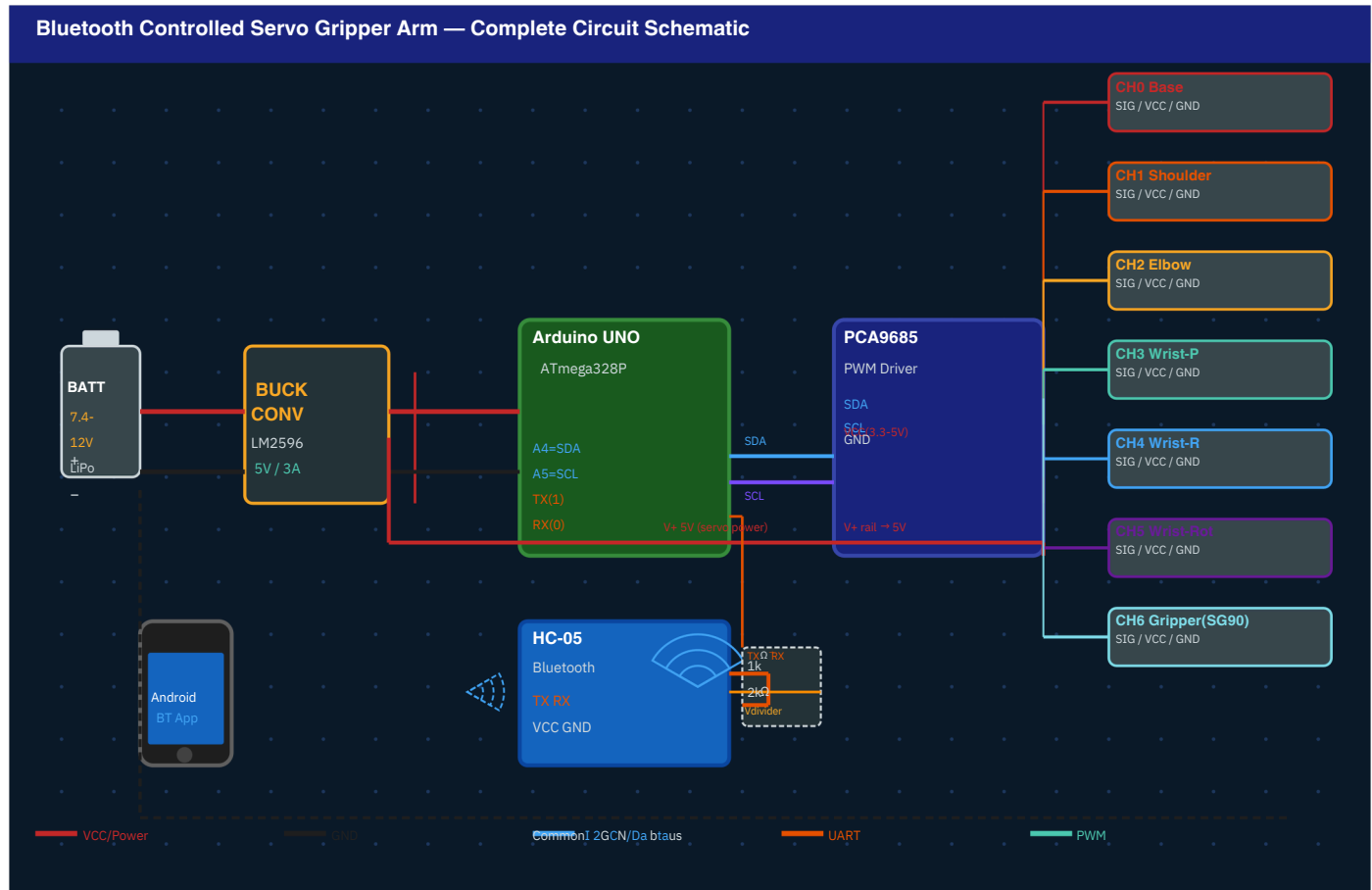
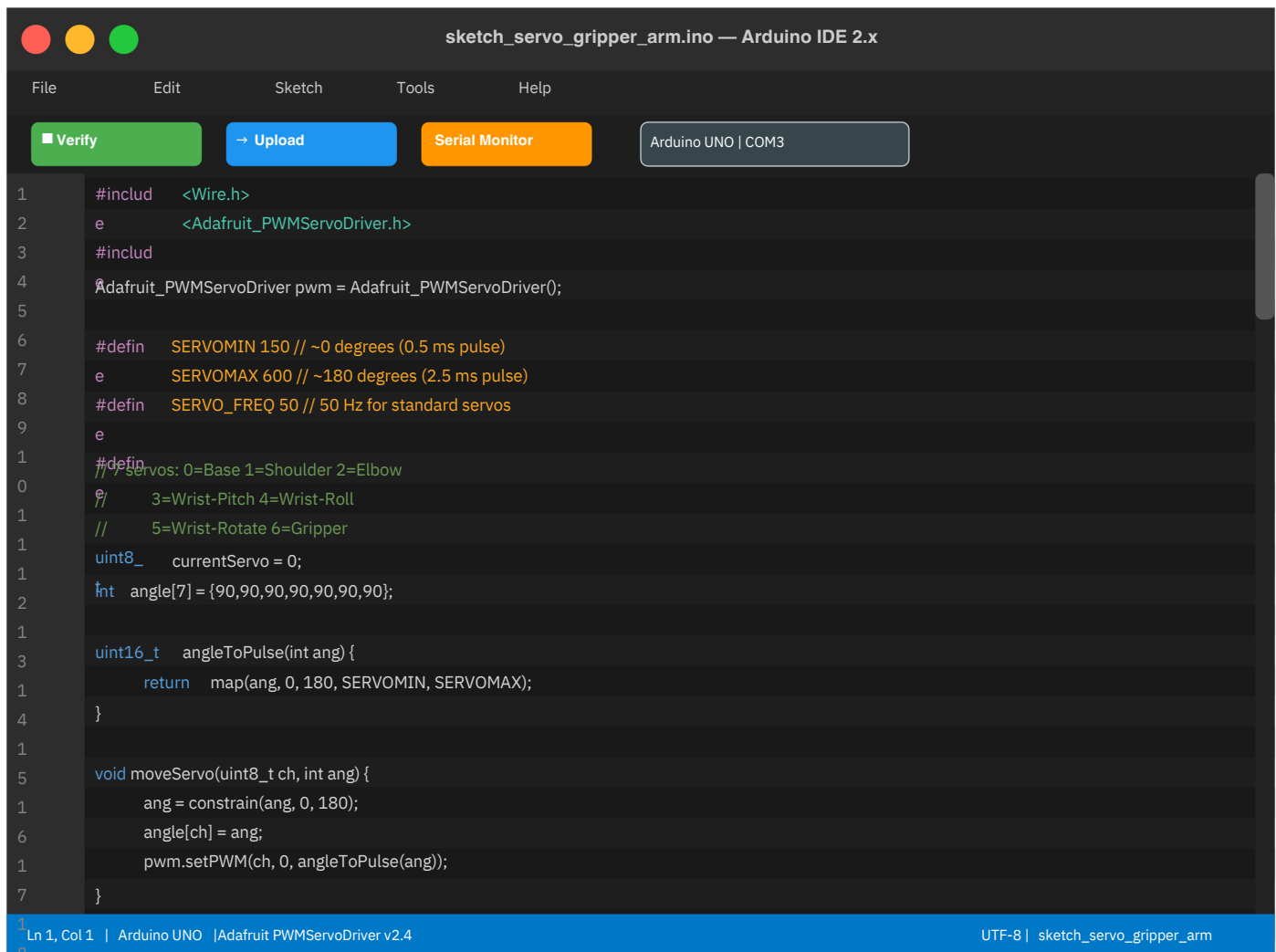


Fig. 2.1 – Complete Circuit Schematic of Bluetooth Controlled Servo Gripper Arm

ARDUINO IDE — SERVO CONTROL SKETCH

The sketch below is displayed inside a simulated Arduino IDE 2.x window with full syntax highlighting. It uses the Adafruit_PWMServoDriver library to control the PCA9685 over I2C and parses single-character Bluetooth commands received on the hardware serial port.



The screenshot shows the Arduino IDE 2.x interface with the file 'sketch_servo_gripper_arm.ino' open. The menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu are buttons for 'Verify', 'Upload', 'Serial Monitor', and a dropdown menu showing 'Arduino UNO | COM3'. The main editor area displays the following C++ code with syntax highlighting:

```
1  #include <Wire.h>
2  #include <Adafruit_PWMServoDriver.h>
3  #include
4  Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();
5
6  #define  SERVOMIN 150 // ~0 degrees (0.5 ms pulse)
7  #define  SERVOMAX 600 // ~180 degrees (2.5 ms pulse)
8  #define  SERVO_FREQ 50 // 50 Hz for standard servos
9
10 #define  Servos: 0=Base 1=Shoulder 2=Elbow
11 //      3=Wrist-Pitch 4=Wrist-Roll
12 //      5=Wrist-Rotate 6=Gripper
13
14 uint8_t  currentServo = 0;
15
16 int  angle[7] = {90,90,90,90,90,90,90};
17
18
19 uint16_t  angleToPulse(int ang) {
20     return  map(ang, 0, 180, SERVOMIN, SERVOMAX);
21 }
22
23 void moveServo(uint8_t ch, int ang) {
24     ang = constrain(ang, 0, 180);
25     angle[ch] = ang;
26     pwm.setPWM(ch, 0, angleToPulse(ang));
27 }
```

The status bar at the bottom indicates 'Ln 1, Col 1 | Arduino UNO | Adafruit_PWMServoDriver v2.4' and 'UTF-8 | sketch_servo_gripper_arm'.

Fig. 2.5 – Arduino IDE 2.x — sketch_servo_gripper_arm.ino

LIST OF FIGURES

<i>Figure No.:</i>	<i>Title</i>	<i>Page No.</i>
Fig.	Arduino UNO Microcontroller Board	4
1.1:	PCA9685 16-Channel Servo Driver Module	4
Fig.	MG996R Servo Motor	5
1.2:	HC-05 Bluetooth Module	5
Fig.	Buck Converter Module	5
1.3:	Robotic Gripper Mechanism	
Fig.	Complete Circuit Schematic of Bluetooth	
Fig. 1.4:	Controlled Servo Gripper Arm	6
Fig. 2.1:	Arduino UNO – PCA9685 I2C Connection	6
Fig. 1.5:	HC-05 Bluetooth Module Wiring	7
Fig. 2.2:	Buck Converter Power Supply Circuit	7
Fig. 1.6:	Assembled Gripper Arm Prototype	9
2.3:		
Fig.		
2.4:		
Fig.		
2.5:		

ABSTRACT

This project presents the design and implementation of a Bluetooth Controlled Servo Gripper Arm using an Arduino UNO microcontroller. The robotic arm consists of six servo motors driven by a PCA9685 16-channel PWM/Servo Driver communicating with the Arduino via the I2C bus. A dedicated gripper mechanism is mounted at the arm's end-effector to enable pick-and-place operations. An HC-05 Bluetooth module provides wireless communication between the arm and an Android smartphone application, allowing real-time control of each joint and the gripper. A buck converter supplies regulated 5V to the servo driver and servo motors, ensuring stable power delivery independent of the Arduino's on-board regulator. The system demonstrates the practical integration of wireless communication, embedded control, and multi-servo coordination in a low-cost robotic platform. The arm can perform precise positional movements in response to Bluetooth commands and is suitable for educational, industrial, and research applications.

CHAPTER 1:

INTRODUCTION

Robotic arms have become indispensable in automation, ranging from large-scale industrial manipulators to small hobby-grade educational systems. A servo-driven gripper arm controlled wirelessly via Bluetooth represents a cost-effective and accessible platform for learning robotics, control systems, and wireless communication. This project designs, builds, and programs a six-degree-of-freedom (6-DOF) robotic arm with an end gripper, entirely controlled from a smartphone over Bluetooth using an Arduino UNO, a PCA9685 servo driver, six servo motors, an HC-05 Bluetooth module, and a buck converter for power management.

1.1: PROBLEM STATEMENT

Manually controlled or tethered robotic arms impose constraints on portability and ease of use. The objective of this project is to develop a compact robotic gripper arm that can be operated wirelessly via Bluetooth from an Android device, providing flexible and intuitive control of six servo joints and a gripper without any physical connection between the controller and the arm.

1.2: LITERATURE SURVEY

Niku (2010) provides a comprehensive foundation in robot kinematics and the use of servo actuators for joint control in manipulators. Choset et al. (2005) discuss principles of robot motion and the importance of degree-of-freedom selection for task-specific robotic arms. Existing Bluetooth-controlled robotic systems—such as those documented by Agarwal et al. (2017)—demonstrate that the HC-05 module in combination with Arduino offers a reliable, low-latency wireless link suitable for real-time servo control. The PCA9685 servo driver, described in the Adafruit library documentation, offloads PWM generation from the microcontroller, enabling simultaneous and jitter-free control of up to 16 servo channels over I2C. Buck converters are widely used in embedded systems (Erickson & Maksimovic, 2001) to efficiently step down voltage, which is critical when powering multiple high-torque servos from a single battery source.

1.3: SYSTEM OVERVIEW

The Bluetooth Controlled Servo Gripper Arm comprises the following major subsystems: a mechanical arm structure with six servo joints and a gripper; an electronic control unit based on Arduino UNO and PCA9685; a wireless interface using the HC-05 Bluetooth module; and a regulated power supply built around a buck converter. Commands are sent from an Android app (e.g., Bluetooth Serial Terminal) to the HC-05 module, decoded by the Arduino, and translated into PWM signals dispatched through the PCA9685 to the appropriate servo motor.

1.4: COMPONENTS USED

1.4.1: Arduino UNO

The Arduino UNO is an open-source microcontroller board based on the ATmega328P. It operates at 5V with a 16 MHz crystal oscillator, features 14 digital I/O pins (6 capable of PWM), 6 analog inputs, and a 32 KB flash memory. In this project it serves as the central processor: it receives serial data from the HC-05 Bluetooth module and communicates with the PCA9685 over the I2C bus (SDA/SCL pins).

1.4.2: PCA9685 Servo Driver

The PCA9685 is a 16-channel, 12-bit PWM/servo driver that communicates via I2C. It can drive up to 16 servos independently at a configurable PWM frequency (typically 50 Hz for standard servos). The module operates from 3.3–5V logic and has a separate V+ power rail for the servo motors, making it ideal for multi-servo applications. Its use frees the Arduino's limited PWM outputs and ensures glitch-free PWM signals to all six servos and the gripper servo simultaneously.

1.4.3: Servo Motors (×6 + Gripper)

Six MG996R metal-gear servo motors provide the rotational joints of the arm. Each MG996R operates at 4.8–7.2V, delivers a stall torque of up to 11 kg·cm at 6V, and has a rotation range of 0°–180°. The gripper uses a lighter SG90 micro servo to open and close the claw. PWM pulse widths in the range of 1 ms–2 ms correspond to 0°–180° rotation.

1.4.4: HC-05 Bluetooth Module

The HC-05 is a master/slave Bluetooth 2.0 serial communication module operating at 3.3V logic with 5V-tolerant RX pin. It communicates with the Arduino via UART at 9600 baud (default). Commands transmitted from the Android app are received by the HC-05 and forwarded to Arduino's hardware serial port for parsing and execution.

1.4.5: Buck Converter

A DC-DC step-down (buck) converter module (e.g., LM2596-based) steps down the 7.4V–12V battery input to a stable 5V–6V output capable of supplying up to 3A continuous current. This dedicated servo power rail prevents the voltage droop that occurs when multiple high-current servos are powered directly from the Arduino's on-board regulator, thereby protecting the microcontroller and ensuring reliable servo operation.

1.4.6: Gripper Mechanism

The end-effector is a two-finger parallel gripper driven by a single SG90 micro servo. A rack-and-pinion or lever-linkage mechanism converts the servo's rotational output into the linear opening/closing motion of the fingers. The gripper can handle objects up to approximately 60 mm in diameter and exert a gripping force sufficient for lightweight pick-and-place tasks.

CHAPTER 2:

CIRCUIT DESIGN AND IMPLEMENTATION

2.1: CIRCUIT DIAGRAM DESCRIPTION

The complete circuit interconnects the following blocks:

Power Supply: A 7.4V LiPo battery (or 9V–12V adapter) feeds the buck converter. The buck converter output (5V–6V, $\geq 3A$) connects to the PCA9685 V+ rail and GND. The Arduino UNO is powered separately via its Vin pin from the same battery, or through USB during programming.

Arduino UNO ↔ PCA9685 (I2C): Arduino A4 (SDA) → PCA9685 SDA; Arduino A5 (SCL) → PCA9685 SCL; Arduino 5V → PCA9685 VCC; Arduino GND → PCA9685 GND. All grounds are common.

PCA9685 ↔ Servos: Channels 0–5 connect to the six MG996R servo motors (Base, Shoulder, Elbow, Wrist-Pitch, Wrist-Roll, Wrist-Rotate). Channel 6 connects to the SG90 gripper servo. Each servo's signal wire (orange/yellow) goes to the PCA9685 channel signal pin; red and brown wires connect to the buck converter's 5V and GND rails respectively.

HC-05 ↔ Arduino: HC-05 VCC → Arduino 5V; HC-05 GND → Arduino GND; HC-05 TX → Arduino RX (Pin 0); HC-05 RX → Arduino TX (Pin 1) through a voltage divider (1k Ω and 2k Ω resistors to protect the 3.3V HC-05 RX pin). Note: disconnect HC-05 TX/RX before uploading code.

2.2: ARDUINO CODE

The following code receives single-character Bluetooth commands and maps them to servo angle increments. The Adafruit_PWMServoDriver library is used to interface with the PCA9685.

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

// PCA9685 at default I2C address 0x40
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

// Servo pulse limits (in PCA9685 counts, 12-bit, 50 Hz)
#define SERVOMIN 150    // ~0 degrees (0.5 ms pulse)
```

```

#define SERVOMAX 600 // ~180 degrees (2.5 ms pulse)
#define SERVO_FREQ 50 // 50 Hz for standard servos

// 7 servos: 0=Base, 1=Shoulder, 2=Elbow,
// 3=Wrist-Pitch, 4=Wrist-Roll, 5=Wrist-
// Rotate, 6=Gripper
uint8_t currentServo = 0; // active servo
int angle[7] = {90,90,90,
                90,90,90,90}; // start at 90 deg

// Convert angle (0-180) to PCA9685 count
uint16_t angleToPulse(int ang) {
    return map(ang, 0, 180, SERVOMIN, SERVOMAX);
}

void moveServo(uint8_t ch, int ang) {
    ang = constrain(ang, 0, 180);
    angle[ch] = ang;
    pwm.setPWM(ch, 0, angleToPulse(ang));
}

void setup() {
    Serial.begin(9600); // HC-05 baud rate
    pwm.begin();
    pwm.setOscillatorFrequency(27000000);
    pwm.setPWMFreq(SERVO_FREQ);
    delay(10);
    // Move all servos to 90 degrees (home)
    for (uint8_t i = 0; i < 7; i++)
        moveServo(i, 90);
    Serial.println("Bluetooth Servo Arm Ready");
}

void loop() {
    if (Serial.available()) {
        char cmd = Serial.read();
        switch (cmd) {
            // Select active servo (0-6)
            case '0': currentServo = 0; break;
            case '1': currentServo = 1; break;
            case '2': currentServo = 2; break;
            case '3': currentServo = 3; break;
            case '4': currentServo = 4; break;
            case '5': currentServo = 5; break;
            case '6': currentServo = 6; break;
            // Increase angle by 5 degrees
            case 'U': moveServo(currentServo,
                                angle[currentServo] + 5); break;
            // Decrease angle by 5 degrees
            case 'D': moveServo(currentServo,
                                angle[currentServo] - 5); break;
            // Open gripper (servo 6 to 0 deg)
            case 'O': moveServo(6, 0); break;
            // Close gripper (servo 6 to 90 deg)
            case 'C': moveServo(6, 90); break;
            // Home all servos

```



```

        case 'H':
            for (uint8_t i = 0; i < 7; i++)
                moveServo(i, 90);
            break;
        default: break;
    }
    // Echo current angle back over Bluetooth
    Serial.print("Servo "); Serial.print(currentServo);
    Serial.print(" -> ");
    Serial.print(angle[currentServo]);
    Serial.println(" deg");
}
}

```

2.3: WORKING PRINCIPLE

On power-up all servos are initialised to 90° (home position). The Arduino listens on its hardware serial port for single-character commands from the HC-05 module. Digits 0–6 select the active servo; 'U' and 'D' increment or decrement the selected servo's angle by 5° per command; 'O' and 'C' open and close the gripper; 'H' returns all joints to the home position. The PCA9685 converts the requested angle into a 12-bit PWM count and drives the corresponding servo channel accordingly. Because the PCA9685 handles all PWM generation autonomously over I2C, the Arduino's CPU is free to process incoming Bluetooth data without timing interruptions.

CHAPTER 3:

ADVANTAGES AND DISADVANTAGES

3.1: Advantages

- Wireless control via Bluetooth eliminates wiring between the controller and arm, improving portability and ease of use.
- The PCA9685 servo driver offloads PWM generation from the Arduino, enabling simultaneous jitter-free control of all six servos and gripper.
- A dedicated buck converter power rail for the servos prevents voltage droop and protects the Arduino's on-board regulator.
- The modular I2C-based design allows easy expansion to more servo channels without changing the Arduino code structure.
- Low cost and readily available components make the system accessible for educational and research purposes.

3.2: Disadvantages

- Bluetooth range is limited to approximately 10 metres, restricting remote operation distance.
- The HC-05 module's UART interface constrains command throughput, making very fast or simultaneous multi-joint movements less smooth.
- MG996R servos consume significant current under load; battery life may be short during continuous heavy-duty operation.
- The open-loop control scheme (no feedback sensors) may result in positional drift over repeated cycles due to servo mechanical play.

CHAPTER 4:

APPLICATIONS OF BLUETOOTH CONTROLLED SERVO GRIPPER ARM

4.1: Educational Robotics:

The arm serves as a practical teaching tool for undergraduate students learning servo control, PWM signals, I2C communication, and Bluetooth interfacing within a single embedded systems project.

4.2: Pick-and-Place Automation:

In small-scale production or laboratory environments the arm can pick lightweight objects and place them at a target location on command, demonstrating basic industrial automation concepts.

4.3: Hazardous Material Handling:

With extended Bluetooth range adapters the arm can handle small objects in environments unsafe for humans, such as radioactive sample handling or chemical laboratory tasks.

4.4: Assistive Technology:

The arm can assist differently-abled individuals in reaching and manipulating objects, controlled wirelessly from an accessible smartphone interface.

4.5: Research and Prototyping:

Researchers can use the platform to prototype and validate inverse kinematics algorithms, computer vision-guided grasping, and multi-robot coordination before scaling to industrial hardware.

CHAPTER 5:

SOFTWARE USED

5.1: Arduino IDE

The Arduino Integrated Development Environment (IDE) is a cross-platform, open-source application used to write, compile, and upload sketches (programs) to Arduino microcontroller boards. Written in Java, the IDE provides a simple editor with syntax highlighting, a serial monitor, and a board/library manager. Version 2.x introduces a language server protocol (LSP) backend for autocompletion and on-the-fly error checking. In this project, the Arduino IDE is used to compile the servo control sketch and upload it to the Arduino UNO via USB.

5.2: Adafruit PWMServoDriver Library

The Adafruit PWMServoDriver library provides a high-level C++ API to control the PCA9685 PWM driver over I2C. Key functions include `pwm.begin()`, `pwm.setPWMFreq()`, and `pwm.setPWM(channel, on, off)`. The library abstracts the 12-bit register writes required to set pulse widths, allowing the developer to work directly in terms of PWM counts mapped from servo angles.

5.3: Bluetooth Serial Terminal (Android App)

A Bluetooth Serial Terminal application (e.g., Serial Bluetooth Terminal by Kai Morich, available on the Google Play Store) is used on an Android smartphone to pair with the HC-05 module and transmit single-character commands. Custom buttons can be configured in the app to send predefined command strings, providing a convenient control interface without requiring custom app development.

CONCLUSION AND FUTURE SCOPE OF WORK

In this project we have successfully designed and implemented a Bluetooth Controlled Servo Gripper Arm using an Arduino UNO, a PCA9685 servo driver, six MG996R servo motors, an SG90 gripper servo, an HC-05 Bluetooth module, and a buck converter for regulated power supply. The system enables wireless real-time control of all six degrees of freedom and the gripper from an Android smartphone over a Bluetooth serial link.

The PCA9685 proved essential for generating stable, simultaneous PWM signals to all channels without burdening the Arduino's processor. The buck converter ensured that the high-current servo loads did not affect microcontroller stability. The Arduino sketch demonstrated efficient command parsing and servo control within the constraints of a single-threaded embedded environment.

Future enhancements may include: integration of potentiometer or encoder feedback for closed-loop position control; a custom Android application with a graphical joystick interface for more intuitive joint manipulation; replacement of Bluetooth with Wi-Fi (ESP8266/ESP32) for extended range and IoT integration; addition of a computer vision module (Raspberry Pi + camera) for autonomous object detection and grasping; and implementation of inverse kinematics to allow Cartesian-space end-effector control rather than joint-space commands.

REFERENCES

1. Niku, S. B. (2010). *Introduction to Robotics: Analysis, Control, Applications* (2nd ed.). Wiley.
2. Choset, H., Lynch, K. M., Hutchinson, S., Kantor, G., Burgard, W., Kavraki, L. E., & Thrun, S. (2005). *Principles of Robot Motion*. MIT Press.
3. Agarwal, P., Mittal, M., & Kalra, P. (2017). Bluetooth controlled robotic arm using Arduino. *International Journal of Engineering Research and Technology*, 6(5), 112–116.
4. Adafruit Industries. (2023). *Adafruit PCA9685 16-Channel PWM/Servo Driver*.
<https://learn.adafruit.com/16-channel-pwm-servo-driver>
5. Erickson, R. W., & Maksimovic, D. (2001). *Fundamentals of Power Electronics* (2nd ed.). Kluwer Academic Publishers.
6. Arduino. (2024). *Arduino UNO Rev3 Datasheet and Reference Manual*.
<https://docs.arduino.cc/hardware/uno-rev3>
7. HC-05 Bluetooth Module Datasheet. (n.d.). ITEad Studio.